

41039 Programming 1

Revision of Week 2: booleans, if, switch and match



Dr. Xuan-Bach Le

le.xuan.bach@hcmut.edu.vn

Faculty of Computer Science and Engineering, HCMUT

September Session 2026

How the next twenty minutes work

Same rules as last week. This is not a test. Nothing here is recorded against you, and nothing you say wrongly costs you anything.

- Every slide **asks** before it **tells**. Commit to an answer first — out loud, or on paper, or just in your head.
- **Answering earns extra credit.** Being wrong out loud earns it too. The only way to miss out is to say nothing.
- Two of you will be at the keyboard near the end. Everyone else earns credit by shouting the fix.

Why this, before loops? Every loop has a condition, and a condition is exactly what Week 2 was about. A loop that tests the wrong `boolean` runs forever, or never.

Recalling something is what fixes it. Re-reading it does not.

Outline

- 1 Types and booleans
- 2 Choosing in Java
- 3 `switch` and fall-through
- 4 Choosing in Python
- 5 Live coding
- 6 Bridge to today

Fill the blanks: types and booleans

- Java is (1) typed: every type is known at (2).
- `10 + 3` has type (3).
`i > 10` has type (4).
- (5) assigns. (6) compares.
- `a && b` is true when (7) sides are.
`a || b` is true when (8) side is.

Word bank

- | | |
|-----------------|-------------|
| a) dynamically | g) = |
| b) boolean | h) run time |
| c) statically | i) both |
| d) String | j) == |
| e) compile time | k) either |
| f) int | l) != |

An if needs a boolean. Almost every rule on this slide follows from that.

Fill the blanks: types and booleans

- Java is (1) typed: every type is known at (2).
- `10 + 3` has type (3).
`i > 10` has type (4).
- (5) assigns. (6) compares.
- `a && b` is true when (7) sides are.
`a || b` is true when (8) side is.

Answers

- (1) c) **statically** (2) e) **compile time**
 (3) f) **int** (4) b) **boolean** — a comparison is an expression, and it has a type
 (5) g) **=** (6) j) **==** — two equals signs
 (7) i) **both** (8) k) **either** — at least one. Python spells them **and**, **or**, **not**

An if needs a boolean. Almost every rule on this slide follows from that.

Which of these compile?

Alone ■ 60 seconds

```
int i = 15;  
  
boolean a = 10 + 3;           // 1  
boolean b = i > 10 && i < 20;  // 2  
int      c = 10 + true;       // 3  
if (i = 15) { ... }           // 4  
boolean d = !(i == 100);      // 5
```

For each line

Compiles, or not? If not — what is the compiler complaining about?

Which of these compile?

Alone ■ 60 seconds

```
int i = 15;

boolean a = 10 + 3;           // 1
boolean b = i > 10 && i < 20;  // 2
int      c = 10 + true;       // 3
if (i = 15) { ... }           // 4
boolean d = !(i == 100);      // 5
```

For each line

Compiles, or not? If not — what is the compiler complaining about?

Answers

1 ✗ int cannot be converted to boolean. Not the number — the **type of the expression**

2 ✓ two booleans joined by && is a boolean

3 ✗ boolean is its own type, not a number. Valid C++; not Java

4 ✗ = assigns, so this is an int, and an if needs a boolean. The classic typo, caught

5 ✓ d is true

Three errors, one idea: expressions have types.

True or false?

Alone ■ 60 seconds



```
int i = 15;

i > 10 && i < 20      // (a)
i < 10 || i > 20      // (b)
!(i == 15)           // (c)
i != 15 || i >= 15    // (d)
i > 15               // (e)
```

And: how would you write (a) in Python?

True or false?

Alone ■ 60 seconds



```
int i = 15;
```

```
i > 10 && i < 20           // (a)
```

```
i < 10 || i > 20           // (b)
```

```
!(i == 15)                 // (c)
```

```
i != 15 || i >= 15         // (d)
```

```
i > 15                     // (e)
```

And: how would you write (a) in Python?

Answers

(a) true — both sides are true

(b) false — *neither* side is

(c) false — `i == 15` is true, and `!` flips it

(d) true — the left side is false, but `||` only needs one

(e) **false** — 15 is not greater than 15. `i >= 15` would be true

Python: `i > 10` and `i < 20`. The words are spelled out

> and >= differ on exactly one value. That value is where the bugs live.

Outline

- 1 Types and booleans
- 2 Choosing in Java**
- 3 `switch` and fall-through
- 4 Choosing in Python
- 5 Live coding
- 6 Bridge to today

Two ifs, again

Alone ■ 60 seconds

```
if (mark < 50) {  
    System.out.println("Fail");  
    mark = mark + 10;    // bonus marks  
}  
if (mark >= 50) {  
    System.out.println("Pass");  
}
```

What prints when mark is...

- A. 45 B. 38 C. 50**
D. One word fixes it. Which, and where?

Two ifs, again

Alone ■ 60 seconds

```
if (mark < 50) {  
    System.out.println("Fail");  
    mark = mark + 10;    // bonus marks  
}  
if (mark >= 50) {  
    System.out.println("Pass");  
}
```

What prints when mark is...

- A. 45 B. 38 C. 50
D. One word fixes it. Which, and where?

Answers

- A. Fail *and* Pass. The first block made it 55, and the second if tests the **new** value
B. Fail only — 48 is still below 50
C. Pass only
D. else — in place of the second if (mark >= 50). Now **exactly one** runs, whatever the first block does to mark

Two ifs give you no guarantee. An else guarantees exactly one.

Order decides the winner

```
int kg = 25;
if (kg > 0) {
    System.out.println("Small: $5");
} else if (kg > 10) {
    System.out.println("Medium: $12");
} else if (kg > 20) {
    System.out.println("Large: $20");
} else {
    System.out.println("Invalid weight");
}
```

Three questions

1. What prints?
2. Which branches can *never* run?
3. Any warning?

Order decides the winner

```
int kg = 25;
if (kg > 0) {
    System.out.println("Small: $5");
} else if (kg > 10) {
    System.out.println("Medium: $12");
} else if (kg > 20) {
    System.out.println("Large: $20");
} else {
    System.out.println("Invalid weight");
}
```

Three questions

1. What prints?
2. Which branches can *never* run?
3. Any warning?

Answers

1. Small: \$5 — for a 25 kg parcel
2. Medium and Large. Anything above 10 is also above 0, and the ladder stops at the **first** true test
3. No. It compiles, runs, and quietly undercharges

✓ **Fix** test `kg > 20` first, then `kg > 10`, then `kg > 0`: **specific before general**

The compiler checks your types. It does not check your logic.

Find the gaps

Pairs ■ 90 seconds



X For some temperatures this prints nothing.

```
if (temp < 20) {  
    System.out.println("Cold");  
}  
else if (temp > 20 && temp < 30) {  
    System.out.println("Warm");  
}  
else if (temp > 30) {  
    System.out.println("Hot");  
}
```

temp is an int. Which values print nothing — and what is the smallest fix?

Find the gaps

Pairs ■ 90 seconds

X For some temperatures this prints nothing.

```
if (temp < 20) {  
    System.out.println("Cold");  
}  
else if (temp > 20 && temp < 30) {  
    System.out.println("Warm");  
}  
else if (temp > 30) {  
    System.out.println("Hot");  
}
```

temp is an int. Which values print nothing — and what is the smallest fix?

Answers

Exactly **20** and exactly **30**. No test is true, and no final else catches them. The fix needs no temp >= 20: the ladder only gets there if temp < 20 was false.

✓ Fix

```
if (temp < 20) {  
    System.out.println("Cold");  
} else if (temp < 30) {  
    System.out.println("Warm");  
} else {  
    System.out.println("Hot");  
}
```

Outline

- 1 Types and booleans
- 2 Choosing in Java
- 3 `switch` and fall-through**
- 4 Choosing in Python
- 5 Live coding
- 6 Bridge to today

Predict the output

Alone ■ 60 seconds

```
switch (day) {  
    case 6 : System.out.println("Saturday");  
    case 7 : System.out.println("Sunday");  
        break;  
    case 1 : System.out.println("Monday");  
    default : System.out.println("Weekday");  
}
```

What prints when day is...

A. 6

B. 7

C. 1

D. 3

Predict the output

Alone ■ 60 seconds

```
switch (day) {  
    case 6 : System.out.println("Saturday");  
    case 7 : System.out.println("Sunday");  
        break;  
    case 1 : System.out.println("Monday");  
    default : System.out.println("Weekday");  
}
```

What prints when day is...

A. 6 B. 7 C. 1 D. 3

Answers

- A. Saturday, Sunday — no break after case 6, so it **falls through** into case 7, then stops at its break
- B. Sunday — it starts *at the match*, not at the top
- C. Monday, Weekday — falls through into default
- D. Weekday — nothing matches, so default runs

A switch starts at the match and stops at a break — not at the next case.

Could this be a switch?

Yes or no — and why?

1. `x == 1 || x == 5 || x == 87`
where `x` is an `int`
2. `mark >= 50`
3. `c == 'y' || c == 'Y'`
where `c` is a `char`
4. `temp == 36.6`
where `temp` is a `double`
5. `choice` is 1, 2 or 3 — anything else is an error

Could this be a switch?

Yes or no — and why?

1. `x == 1 || x == 5 || x == 87`
where `x` is an `int`
2. `mark >= 50`
3. `c == 'y' || c == 'Y'`
where `c` is a `char`
4. `temp == 36.6`
where `temp` is a `double`
5. choice is 1, 2 or 3 — anything else is an error

Answers

1. ✓ equality on an `int`: stack case 1 :
case 5 : case 87 : over one body
2. ✗ not an equality check — no ranges
3. ✓ `char` is switchable: case `'y'` :
case `'Y'` :
4. ✗ `double` is not switchable. Only `byte`, `short`, `char`, `int`, `String` (and `enums`, later)
5. ✓ three cases; the error goes in `default`

switch is a restricted tool: equality only, on a short list of types.

Outline

- 1 Types and booleans
- 2 Choosing in Java
- 3 `switch` and fall-through
- 4 Choosing in Python**
- 5 Live coding
- 6 Bridge to today



X Five things are wrong. Someone learnt Java first.

```
1 x = int(input("Enter a number: "))
2 if (x > 0 && x < 10) {
3     print("small")
4 else if x == 10:
5     print("ten")
6 else
7     print("big")
```

✗ Five things are wrong. Someone learnt Java first.

```
1 x = int(input("Enter a number: "))
2 if (x > 0 && x < 10) {
3     print("small")
4 else if x == 10:
5     print("ten")
6 else
7     print("big")
```

✓ Fixed

```
x = int(input("Enter a number: "))
if x > 0 and x < 10:
    print("small")
elif x == 10:
    print("ten")
else:
    print("big")
```

- Line 2: and, not &&; a **colon**, not {
- Line 4: elif, not else if
- Line 5: the body must be **indented** — indentation *is* the block
- Line 6: else needs its colon

The brackets around the condition are allowed. They are just not Python style.

match: predict, then compare

```
match day:
    case 6 | 7:
        print("Weekend")
    case 1:
        print("Monday")
    case _:
        print("Weekday")
```

Three questions

1. `day = 1`: what prints? (Java's switch printed two lines for 1.)
2. `case 6 | 7` and `case _`: what are they in Java?
3. Delete `case _`. What prints for `day = 9`?

match: predict, then compare

```
match day:
    case 6 | 7:
        print("Weekend")
    case 1:
        print("Monday")
    case _:
        print("Weekday")
```

Three questions

1. `day = 1`: what prints? (Java's switch printed two lines for 1.)
2. `case 6 | 7` and `case _`: what are they in Java?
3. Delete `case _`. What prints for `day = 9`?

Answers

1. Monday only. **No fall-through**: only the matching block runs, so there is no break to forget
2. Stacked labels `case 6 :` `case 7 :`, and default `:`
3. **Nothing**, and no error. Handle the case you are sure cannot happen — it costs two lines

	Java switch	Python match
Fall-through	yes — needs break	no — never
Grouping	stacked case labels	<code>case a b:</code>
Default	<code>default :</code>	<code>case _:</code>

Outline

- 1 Types and booleans
- 2 Choosing in Java
- 3 `switch` and fall-through
- 4 Choosing in Python
- 5 Live coding**
- 6 Bridge to today

```
import java.util.Scanner;

public class Ticket {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Age: ");
        int age = input.nextInt();

        // TODO 1: "Invalid age" if < 0 or > 120
        //      -- test this FIRST. Why?
        // TODO 2: "Child: $8"    if under 18
        // TODO 3: "Senior: $10"  if 65 or over
        // TODO 4: "Adult: $15"   otherwise

    }
}
```

Run it with all of these

17	→	Child: \$8
18	→	Adult: \$15
64	→	Adult: \$15
65	→	Senior: \$10
-1	→	Invalid age
130	→	Invalid age

Exactly **one** line, for every age.
One volunteer types; the room
shouts fixes — credit for both.

One answer — and three ways to get it wrong

✓ One way

```
if (age < 0 || age > 120) {  
    System.out.println("Invalid age");  
} else if (age < 18) {  
    System.out.println("Child: $8");  
} else if (age >= 65) {  
    System.out.println("Senior: $10");  
} else {  
    System.out.println("Adult: $15");  
}
```

One answer — and three ways to get it wrong

✓ One way

```
if (age < 0 || age > 120) {  
    System.out.println("Invalid age");  
} else if (age < 18) {  
    System.out.println("Child: $8");  
} else if (age >= 65) {  
    System.out.println("Senior: $10");  
} else {  
    System.out.println("Adult: $15");  
}
```

Each of these compiles. What goes wrong?

- A. `age < 0 && age > 120`
- B. `age > 65` instead of `age >= 65`
- C. test `age < 18` before the invalid test

One answer — and three ways to get it wrong

✓ One way

```
if (age < 0 || age > 120) {
    System.out.println("Invalid age");
} else if (age < 18) {
    System.out.println("Child: $8");
} else if (age >= 65) {
    System.out.println("Senior: $10");
} else {
    System.out.println("Adult: $15");
}
```

Each of these compiles. What goes wrong?

- A. `age < 0 && age > 120`
- B. `age > 65` instead of `age >= 65`
- C. test `age < 18` before the invalid test

Answers

A. Never true: no age is below 0 *and* above 120. -1 prints Child, 130 prints Senior

B. A 65-year-old pays Adult. Off by one

C. -1 is under 18 too: Child: \$8

All three compile and run. Only the boundary inputs catch them — which is why the six test inputs were 17/18, 64/65, and two that “can never happen”.

Live coding: which day?

A different volunteer ■ 4 min

```
day = input("Day number (1-7): ")

# TODO 1: day is text right now.
#         Turn it into a whole number.

# TODO 2: one match statement that prints
#         Weekend           for 6 or 7
#         Weekday           for 1 to 5
#         Invalid day       for anything else
#         careful -- Python has no break,
#         and no stacked cases
```

Run it with all of these

6 → Weekend
3 → Weekday
9 → Invalid day

Before typing: what will look different from a Java switch?

Live coding: which day?

A different volunteer ■ 4 min

```
day = input("Day number (1-7): ")

# TODO 1: day is text right now.
#         Turn it into a whole number.

# TODO 2: one match statement that prints
#         Weekend           for 6 or 7
#         Weekday           for 1 to 5
#         Invalid day       for anything else
#         careful -- Python has no break,
#         and no stacked cases
```

Run it with all of these

6 → Weekend
3 → Weekday
9 → Invalid day

Before typing: what will look different from a Java switch?

Watch for

Skip TODO 1 and every input prints Invalid day, with no error: the text "6" never equals the number 6.

One answer — and the same thing in Java

✓ Python

```
day = int(day)

match day:
    case 6 | 7:
        print("Weekend")
    case 1 | 2 | 3 | 4 | 5:
        print("Weekday")
    case _:
        print("Invalid day")
```

Type six: a ValueError at int() — Week
1, still true.

One answer — and the same thing in Java

✓ Python

```
day = int(day)

match day:
    case 6 | 7:
        print("Weekend")
    case 1 | 2 | 3 | 4 | 5:
        print("Weekday")
    case _:
        print("Invalid day")
```

Type six: a ValueError at int() — Week 1, still true.

Java: count the differences

```
switch (day) {
    case 6 :
    case 7 :
        System.out.println("Weekend");
        break;
    case 1 : case 2 : case 3 :
    case 4 : case 5 :
        System.out.println("Weekday");
        break;
    default :
        System.out.println("Invalid day");
}
```

Stacked labels for |, default for _ — and two breaks. Delete the first: 6 prints Weekend *and* Weekday.

Outline

- 1 Types and booleans
- 2 Choosing in Java
- 3 `switch` and fall-through
- 4 Choosing in Python
- 5 Live coding
- 6 Bridge to today

Where Week 2 stops

Everything you can write so far

```
int age = input.nextInt();  
  
if (age < 0 || age > 120) {  
    System.out.println("Invalid age");  
}  
else ...
```

The user types -5. The program says
Invalid age — and ends.

The question for today

How would you *keep asking* until the age is valid?

Where Week 2 stops

Everything you can write so far

```
int age = input.nextInt();  
  
if (age < 0 || age > 120) {  
    System.out.println("Invalid age");  
}  
else ...
```

The user types -5. The program says
Invalid age — and ends.

The question for today

How would you *keep asking* until the age
is valid?

✓ After today

```
int age;  
do {  
    System.out.print("Age: ");  
    age = input.nextInt();  
} while (age < 0 || age > 120);
```

The same boolean as the if, now deciding
whether to go *round again*.

State, then choosing. Today: repeating — the third tool.

You should now be able to...

Week 2

- ✓ say the **type of an expression**, and why `if (x = 5)` does not compile
- ✓ evaluate `&&`, `||` and `!` — and tell `>` from `>=` on the boundary
- ✓ explain why **two ifs are not an if-else**
- ✓ order an `else if` ladder: **specific before general**
- ✓ predict a switch with a missing `break`, and use **fall-through on purpose**
- ✓ say when a switch is *not* available
- ✓ write the same choice with `if/elif/else` and `match` — and say why `match` needs no `break`
- ✓ ask of every branch: **what if it is exactly the boundary?**

If any box is unticked

Sections 01–04, quiz questions 1–8 and exercises 1–3:

lexuanbach.github.io/41039